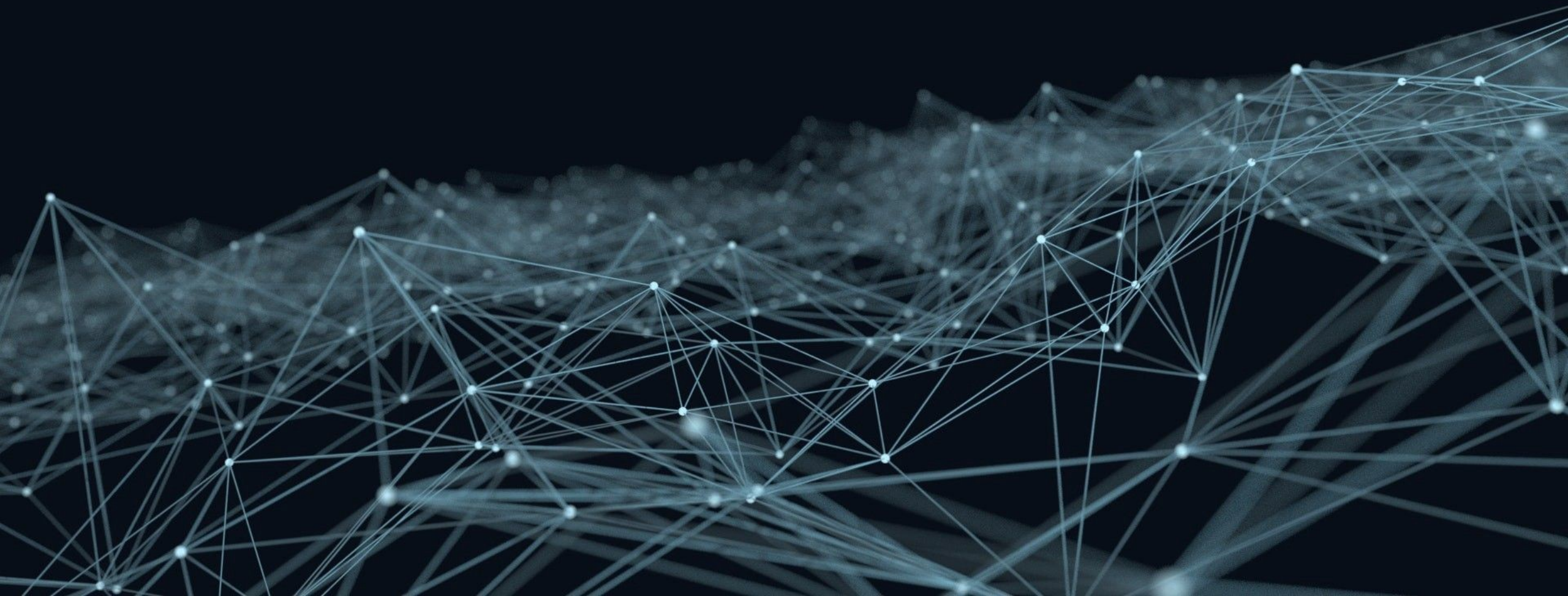
Java Full Stack Development

1. Streams and Functional Programming



Module Topics

- Why Functional Style Matters in Enterprise Java
- Lambdas and Method References
- Stream Pipeline Architecture
- Collectors and Assembling Results
- Handling Nested and Absent Values
- Performance Considerations

Java Design

- Java was designed for a different world
  - Designed for
    - Applications that runs on a single core
    - Data sets that fit comfortably in memory
    - Data processing common use case is reading a database record and displaying it on screen
  - Still works in today's world but forces the programmer to write a great deal of boilerplate code
    - Written to manage how the computer executes
    - Not providing a specification for what the computer is supposed to do, for example, like SQL

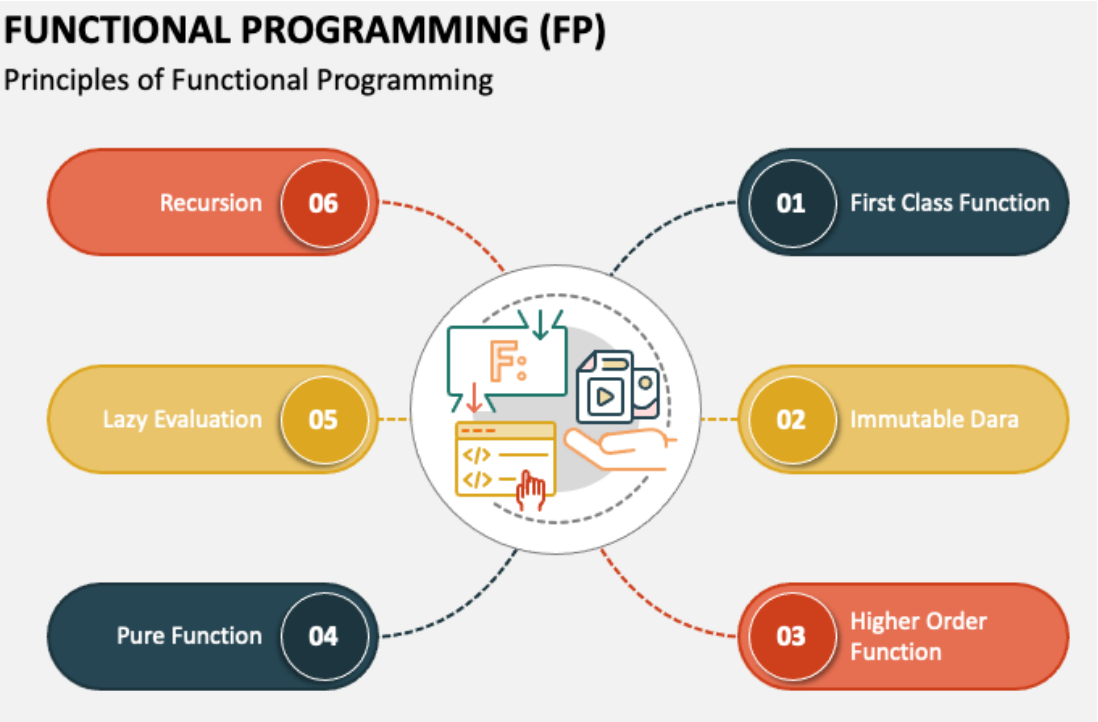


Image Credit: https://www.sketchbubble.com/en/presentation-functional-programming-fp.html

Modern Programming

- The world enterprise software runs in today
  - Services process millions of events per hour from queues, APIs, and sensors
  - Multi-core CPUs are universal but most sequential code uses exactly one core
  - Data pipelines transform, filter, and aggregate collections continuously
  - Distributed systems produce streams of data that must be processed as they arrive
  - Latency and throughput are first-class requirements, not afterthoughts

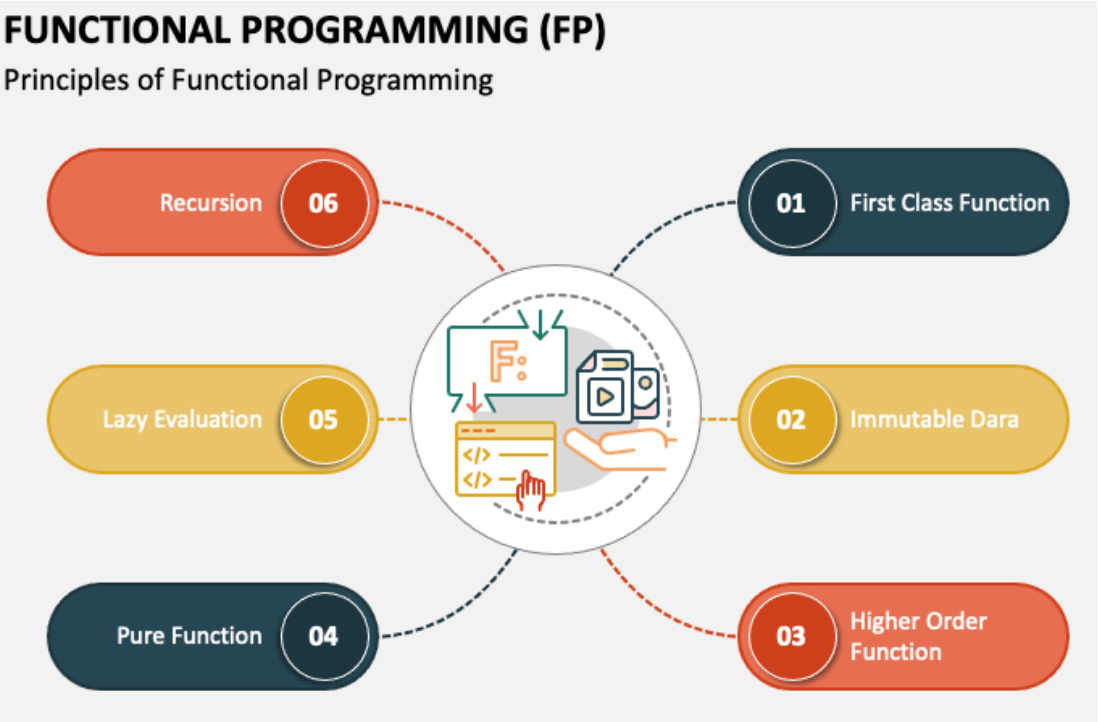


Image Credit: https://www.sketchbubble.com/en/presentation-functional-programming-fp.html

The Multi-Core Problem

- Before 2005, each new processor generation was faster
  - After 2005, each generation has more cores
  - A modern server CPU has 32, 64, or 128 cores
  - A developer's laptop has 8 or 16
- For a simple loop
  - Loop variable advances one step at a time
  - Each iteration depends on the previous iterator state (loop variable)
  - Nothing in the code tells the runtime it is safe to run iterations concurrently
  - Result: 63 out of 64 cores sit idle while the loop runs

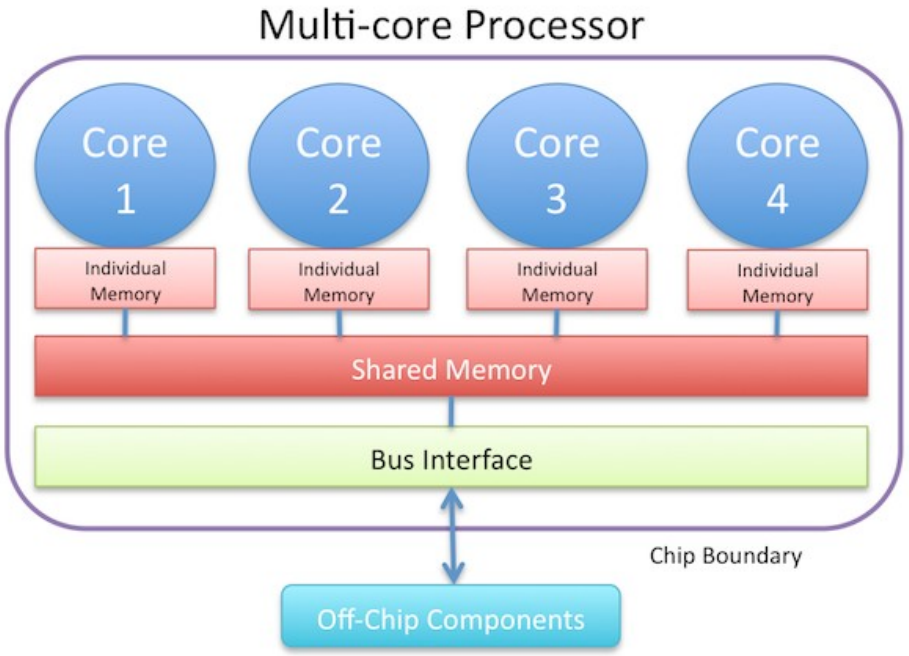


Image Credit: https://www.microcontrollertips.com/terminology-multicore-processors/

Functional Programming

- Functional programs separate what to compute from how to execute it
  - A pure function can be applied independently to each element of a collection
    - Can run on any number of cores simultaneously
    - Because there is no shared mutable state between invocations
  - Allows parallelism but parallel execution requires the right conditions
    - Large data sets
    - CPU-bound work
    - Minimal or no I/O
  - Functional code can be parallelized but stateful code like imperative loops generally cannot be

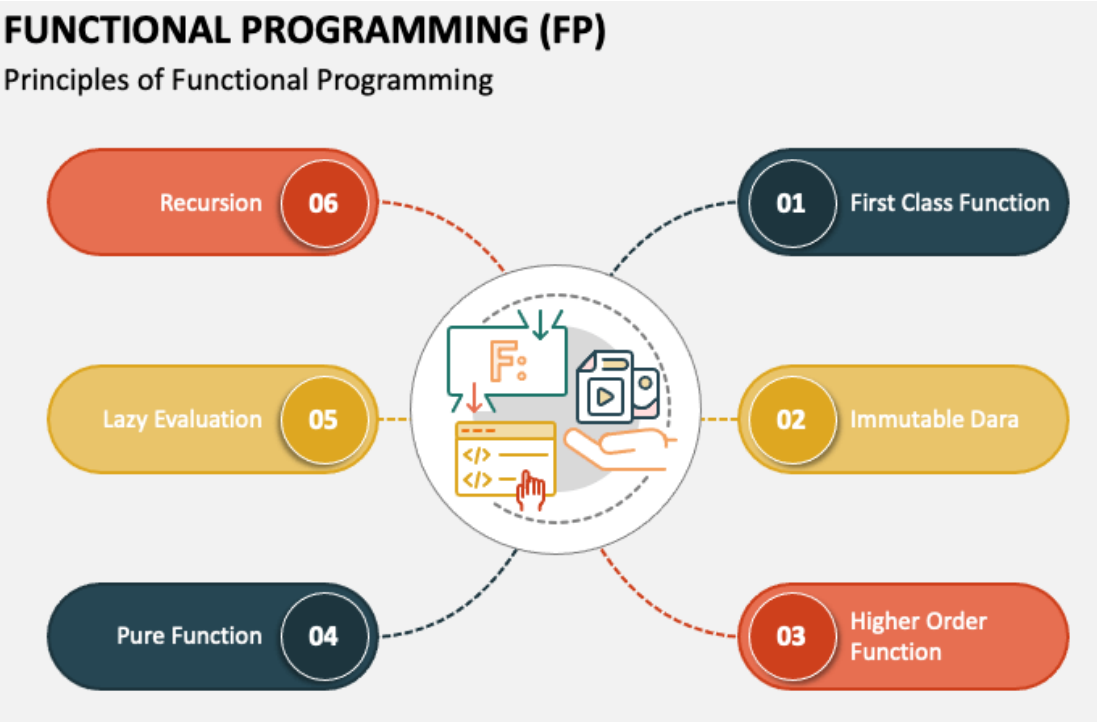


Image Credit: https://www.sketchbubble.com/en/presentation-functional-programming-fp.html

The Data Volume Problem

- What "processing a collection" looked like in 2000
  - A List<Customer> with 500 records. Load all, loop, display
- What "processing a collection" looks like today

Source	Characteristics	Traditional Loop Suitability
Database result set (millions of rows)	Too large to load into memory at once	Poor — requires pagination, cursor management
Kafka topic (continuous event stream)	Unbounded — never fully "loaded"	Not applicable — there is no final list
REST API with pagination (external data)	Arrives in chunks over time	Poor — requires stateful accumulation across calls
In-memory aggregation (analytics query)	Large collections, multiple transformation stages	Works, but produces deeply nested, stateful loop code
Sensor / IoT telemetry feed	High-frequency, potentially infinite	Not applicable — stream processing required

  - In each case, the programmer wants to express a transformation pipeline
    - Filter this, transform that, group by this, take the top N, etc.
    - Without worrying about how data arrives, how much of it there is, or how many cores are available to process it

Java Functional Programming

- Functions as values
  - Functions are first class citizens
  - A function can be
    - Stored in a variable
    - Passed as a parameter
    - Returned from another function
    - A function is just another type of data
  - Lambdas provide a syntax for function literals

```
// A function stored as a value
Predicate<Employee> isActive = e -> e.status() == Status.ACTIVE;

// Passed as a parameter
employees.stream().filter(isActive).collect(Collectors.toList());
```

Java Functional Programming

- Pure functions
  - A pure function's output depends only on its inputs
  - It has no side effects
  - It does not modify external state, write to a database, or change a field on an object
  - Given the same input, it always returns the same output
- Immutability as the default
  - Rather than modifying an existing object
    - Functional code creates a new object with the changed value
    - Collections are not mutated, transformations produce new collections

```
// Pure — output depends only on the input; nothing external is changed
Function<Employee, String> fullName = e -> e.firstName() + " " + e.lastName();

// Impure — modifies external state; output varies with external conditions
Function<Employee, String> fullName = e -> {
    auditLog.record(e.id());   // side effect
    return e.firstName() + " " + e.lastName();
};
```

Java Functional Programming

- Java is not a functional language
- Java is a multi-paradigm language
  - Supports functional style through lambdas, method references, the Stream API, and Optional
  - Does not enforce immutability or purity
  - Writing functional-style code in Java is the programmer's responsibility
    - The language provides the tools
    - The engineering culture provides the discipline

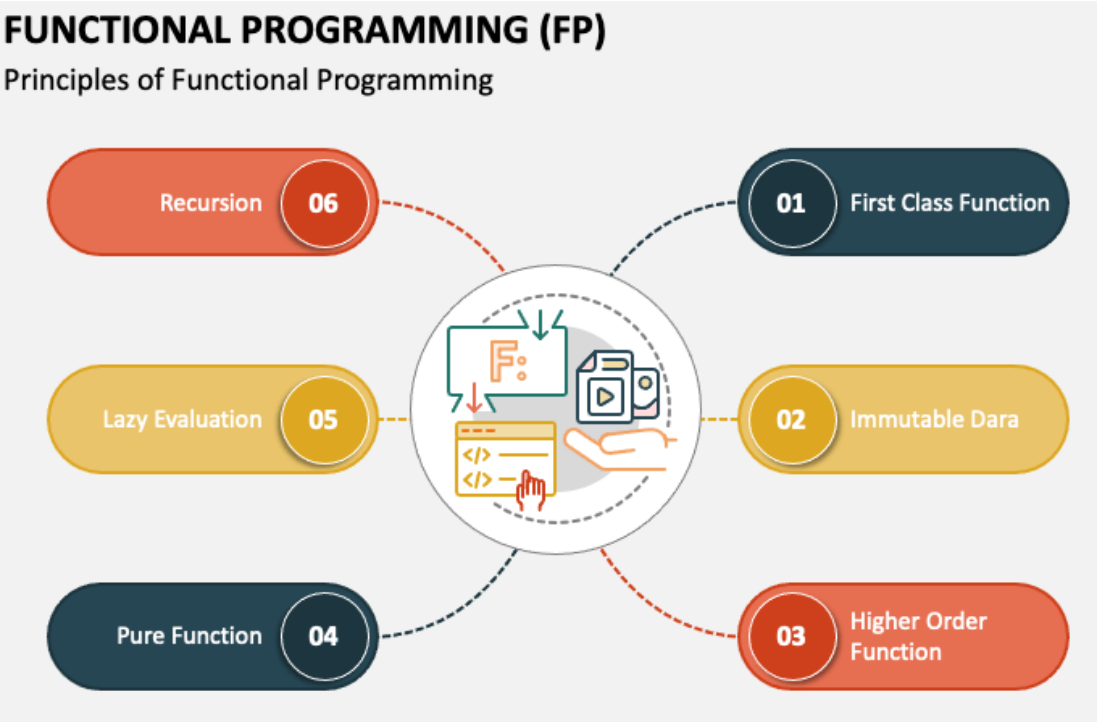


Image Credit: https://www.sketchbubble.com/en/presentation-functional-programming-fp.html

Java Functional Programming

- Java's adoption of functional ideas was deliberate and phased

Year	Java Version	What Changed	Significance
1995	Java 1.0	Anonymous classes	First way to pass behaviour — verbose, but the concept existed
2004	Java 5	Generics, enhanced `for` loop	Type-safe collections; iteration became slightly less painful
2011	Java 7	Fork/Join framework	First standard API for parallel work — but required manual decomposition
2014	**Java 8**	**Lambdas, Stream API,** `Optional` **,** `java.util.function`	**The functional pivot. The most significant Java release in a decade.**
2016	Java 9	`Stream.takeWhile` , `dropWhile` , `iterate` improvements	Stream API matured
2018	Java 11 (LTS)	`var` in lambdas, collection factory improvements	Minor friction reductions
2021	Java 17 (LTS)	Records, sealed classes, pattern matching	Functional data modelling — immutable value types by default
2023	Java 21 (LTS)	Virtual threads, `SequencedCollection`	Concurrency model aligned with functional, non-blocking idioms

Influence of Other Languages

- Big data, reactive systems, and microservices in the 2010s created real production pressure
  - Frameworks like Apache Spark (2014), Reactor, and RxJava were built entirely on functional pipeline models
  - Enterprise Java developers working with these systems needed the same vocabulary in the core language

Language	Key Influence on Java
Haskell (1990)	Demonstrated that pure functional programming at scale was practical. Lazy evaluation, type inference, and the concept of monads (which `Optional` loosely resembles) originated here.
Scala (2004)	Ran on the JVM, combining object-oriented and functional styles. Proved that functional idioms were compatible with the Java ecosystem. Many Java 8 design decisions were informed by Scala's adoption patterns.
Clojure (2007)	Brought Lisp-style functional programming to the JVM. Demonstrated the value of immutable data structures for concurrency.
C# LINQ (2007)	`IEnumerable<T>.Where(...).Select(...).OrderBy(...)` — the direct precedent for `stream().filter(...).map(...).sorted(...)`. Proved enterprise developers could adopt query-style collection processing in a mainstream OO language.
Python generators / list comprehensions	Popularised lazy, pipeline-style data processing outside the academic world.

Declarative vs Imperative

- Programming paradigms can be characterized by what the programmer needs to specify
  - Imperative programming specifies how to perform a task
    - Create an empty list
    - For each element in the source, check condition A, then check condition B
    - If both pass, transform the element and add it to the list
    - Sort the list by field X
    - Return the list

Dimension	Imperative	Declarative
Focus	Execution steps	Result specification
Mutable state	Explicit, programmer-managed	Absent — transformations produce new values
Parallelism	Impossible without restructuring	Possible — runtime can choose execution strategy
Readability	Describes *how* — reader must reconstruct *what*	Describes *what* — intent is immediately visible
Testability	Depends on shared state — harder to isolate	Each transformation is a pure function — easy to test in isolation
SQL analogy	Stored procedure with cursor loops	`SELECT ... WHERE ... ORDER BY ...`

Declarative vs Imperative

- Declarative programming: specify what the process should do
  - From this collection, I want the elements that satisfy condition A and B
  - Transformed by function F
  - Sorted by field X

Dimension	Imperative	Declarative
Focus	Execution steps	Result specification
Mutable state	Explicit, programmer-managed	Absent — transformations produce new values
Parallelism	Impossible without restructuring	Possible — runtime can choose execution strategy
Readability	Describes *how* — reader must reconstruct *what*	Describes *what* — intent is immediately visible
Testability	Depends on shared state — harder to isolate	Each transformation is a pure function — easy to test in isolation
SQL analogy	Stored procedure with cursor loops	`SELECT ... WHERE ... ORDER BY ...`

The Streams API

- A lazily-evaluated pipeline of transformation operations applied to a sequence of elements, producing a result when a terminal operation is invoked

Misconception	Reality
"A stream is a type of collection"	A stream is a *pipeline over* a collection (or other source). It holds no data itself. It is consumed once and then gone.
"Streams are always faster than loops"	For small collections (< ~1,000 elements), streams are marginally *slower* due to lambda dispatch and pipeline setup overhead. Speed is not the point.
" `.parallelStream()` automatically makes code faster"	Parallel streams use the shared ForkJoinPool. I/O-bound work in a parallel stream will deadlock or degrade the entire application. Parallelism requires specific conditions.
"Streams replace collections"	Collections are data structures — they store and retrieve values. Streams are processing pipelines — they transform sequences. Both are necessary; they serve different purposes.
"Functional programming means no `for` loops ever"	Loops remain the right tool for genuinely iterative, stateful algorithms. The discipline is choosing the right tool: stream pipelines for transformations, loops for stateful iteration.
"Streams are a new idea in Java"	The Stream API formalised patterns that Java developers had been building manually with `Iterator`, anonymous classes, and `Comparator` for fifteen years. The API made the pattern first-class.

Imperative Iteration

- A typical service method using a for loop
  - Mixes four distinct concerns in one block
- The deeper problem
  - Imperative loops are inherently sequential and stateful
  - No part of the computation can be moved, delayed, or parallelized without understanding all the others

Concern	Problem at Scale
Iteration mechanics	Non-parallelisable; loop variable and result list are shared mutable state
Filtering (business rules)	Logic buried inside the loop — hard to test in isolation
Transformation (mapping)	Co-located with iteration — can't be reused without extracting to a method
Sorting	Post-hoc second pass — easy to forget; silently produces wrong order on empty result

The Declarative Shift

- Describing results, not procedures
- Stream API inverts the relationship
  - Dev write a specification of what the result should look like
  - The JVM decides how to produce it
- Same design approach as SQL
  - SQL specifies what is to be done
  - The database engine figures out how to execute the query

```
// Each line corresponds to exactly one concern — separated and named
return employees.stream()
    .filter(e -> e.status() == Status.ACTIVE)          // concern: filter
    .filter(e -> e.salary().compareTo(THRESHOLD) > 0)  // concern: filter
    .sorted(Comparator.comparing(Employee::name))      // concern: sort
    .map(employeeMapper::toDto)                        // concern: transform
    .collect(Collectors.toUnmodifiableList());          // concern: collect
```

The Streams API

- Streams API does not make Java faster
  - Stream pipelines are generally no faster than equivalent well-written loops for sequential, small-to-medium collections
  - Streams were designed to support program correctness, not to improve execution performance

What the Stream API was designed for	What it was NOT designed for
Separating concerns — filter, transform, sort, collect are distinct stages	Micro-optimisation of small loops
Eliminating shared mutable state — no result list to corrupt	Replacing all uses of `for` loops
Composable, type-safe vocabulary for collection transformations	Making I/O-bound operations faster
Enabling lazy evaluation and short-circuit termination	Being faster than loops on < 100 elements

When Not to Use Streams

Scenario	Better Approach
Building an index or lookup map from scratch	Traditional loop or `Map.computeIfAbsent()` — construction logic is inherently stateful
Algorithm where each step depends on the previous result	Loop — stateful iteration is what loops are designed for
Checked exceptions inside lambdas	Wrap in an unchecked utility method, or use a loop — `try-catch` inside lambdas obscures pipelines
Modifying elements of a collection in-place	Traditional loop — streams create new collections, they do not mutate existing ones
Short collections where a pipeline reads worse than a direct call	Direct call or `if-else` — a 2-element stream pipeline is not more readable than an `if` statement
Debugging complex intermediate state step-by-step	Loop with logging — debugger navigation is simpler; `peek()` is a workaround, not a production tool

Lambdas and Functional Interfaces

- A functional interface is an interface with exactly one abstract method
- A lambda is an instance of a functional interface
  - The syntax is shorthand for an anonymous implementation
- Target typing
  - The compiler resolves parameter types from the declared type on the left-hand side
  - The same lambda body can implement different functional interfaces

```
// Four equivalent expressions — all produce Comparator<Employee>

// 1. Anonymous class (pre-Java 8 baseline)
Comparator<Employee> c1 = new Comparator<Employee>() {
    public int compare(Employee a, Employee b) {
        return a.name().compareTo(b.name());
    }
};

// 2. Lambda — explicit types
Comparator<Employee> c2 = (Employee a, Employee b) -> a.name().compareTo(b.name());

// 3. Lambda — inferred types (preferred)
Comparator<Employee> c3 = (a, b) -> a.name().compareTo(b.name());

// 4. Method reference — delegate to existing method
Comparator<Employee> c4 = Comparator.comparing(Employee::name);
```

Lambdas and Functional Interfaces

- The single abstract method rule
  - Any interface with exactly one abstract method is a functional interface
  - @FunctionalInterface is a compile-time safety annotation
    - It fails the build if a second abstract method is added
  - Default and static methods do not count
    - Comparator<T> has many default methods
    - It is still a functional interface because it has exactly one abstract method: compare(T, T)

```java
@FunctionalInterface
public interface Formatter {

    // The single abstract method — what a lambda implements
    String format(String input);

    // Default method — concrete, inherited, does not break the SAM rule
    default Formatter andThen(Formatter next) {
        return input -> next.format(this.format(input));
    }

    // Static factory — belongs to the type, not to any instance
    static Formatter upperCase() {
        return input -> input.toUpperCase();
    }
}
```

```java
// Lambda assigned to the abstract method
Formatter trim    = input -> input.trim();
Formatter shout   = Formatter.upperCase();         // static factory
Formatter trimAndShout = trim.andThen(shout);      // default method

trimAndShout.format("  hello world  ");            // → "HELLO WORLD"
```

Lambdas and Functional Interfaces

- The single abstract method rule
  - With exactly one abstract method
    - The compiler binds the lambda body unambiguously
    - Two abstract methods would make the binding ambiguous
  - Enforces Interface Segregation
    - A functional interface represents one behaviour
    - Predicate<T> tests one thing.
    - Function<T,R> transforms one thing
    - Consumer<T> consumes one thing

```java
@FunctionalInterface
public interface Formatter {

    // The single abstract method — what a lambda implements
    String format(String input);

    // Default method — concrete, inherited, does not break the SAM rule
    default Formatter andThen(Formatter next) {
        return input -> next.format(this.format(input));
    }

    // Static factory — belongs to the type, not to any instance
    static Formatter upperCase() {
        return input -> input.toUpperCase();
    }
}
```

```java
// Lambda assigned to the abstract method
Formatter trim    = input -> input.trim();
Formatter shout   = Formatter.upperCase();         // static factory
Formatter trimAndShout = trim.andThen(shout);      // default method

trimAndShout.format("  hello world  ");            // → "HELLO WORLD"
```

Core Functional Interfaces

- From java.util.function

Interface	Signature	Typical Stream Use
`Function<T,R>`	`R apply(T t)`	`.map(mapper::toDto)`
`Predicate<T>`	`boolean test(T t)`	`.filter(e -> e.status() == ACTIVE)`
`Consumer<T>`	`void accept(T t)`	`.forEach(auditService::log)`
`Supplier<T>`	`T get()`	`.orElseThrow(EntityNotFoundException::new)`
`BiFunction<T,U,R>`	`R apply(T t, U u)`	`.collect()` downstream combiners
`UnaryOperator<T>`	`T apply(T t)`	`.map(String::toUpperCase)`
`BinaryOperator<T>`	`T apply(T t1, T t2)`	`.reduce(BinaryOperator)`
`Comparator<T>`	`int compare(T o1, T o2)`	`.sorted(Comparator.comparing(...))`

Method References

Form	Syntax — When to Use
Static method reference	`ClassName::staticMethod` — when the lambda calls a static method: `Integer::parseInt` , `Objects::nonNull`
Instance method of a particular instance	`instance::method` — when you already have the target object: `employeeMapper::toDto` , `log::info`
Instance method of an arbitrary instance	`ClassName::instanceMethod` — when the stream element IS the receiver: `Employee::name` , `String::toUpperCase`
Constructor reference	`ClassName::new` — when the lambda creates an instance: `ArrayList::new` , `EmployeeDto::new`

```
// Form 2: employeeMapper is a specific object. Call toDto ON it.
employees.stream().map(employeeMapper::toDto)  // same as: e -> employeeMapper.toDto(e)

// Form 3: Employee::name — the stream element IS the receiver.
employees.stream().map(Employee::name)         // same as: e -> e.name()
```

Method References

- Static method reference
  - ClassName::staticMethod
  - The lambda calls a static method on a class
  - The stream element becomes the argument
  - The receiver is the class itself, not an object
  - The stream element is passed in as the argument to the static method
  - If you would write
    - ClassName.method(element)
  - as a static call, the reference is ClassName::method

```
// Equivalent lambda
Function<String, Integer> parse = s -> Integer.parseInt(s);

// Method reference – Integer is the class, parseInt is its static method
Function<String, Integer> parse = Integer::parseInt;

// The stream element (a String) is passed as the argument to parseInt
List<Integer> ids = List.of("1", "2", "3")
    .stream()
    .map(Integer::parseInt)     // same as: s -> Integer.parseInt(s)
    .collect(Collectors.toList());
```

```
.filter(Objects::nonNull)          // s -> Objects.nonNull(s)
.map(String::valueOf)              // n -> String.valueOf(n)
.filter(Objects::isNull)           // s -> Objects.isNull(s)
```

Method References

- Instance method of an Instance
  - object::instanceMethod
  - The lambda calls a method on one specific, already-existing object
  - The stream element becomes the argument
  - You have an object in hand
  - The lambda calls a method on that object, passing the stream element as the argument
  - If you would write
    - myObject.method(element)
  - The reference is myObject::method

```
// You have a specific mapper object
EmployeeMapper mapper = new EmployeeMapper();

// Equivalent lambda — mapper is captured from the enclosing scope
Function<Employee, EmployeeDto> toDto = e -> mapper.toDto(e);

// Method reference — mapper is the specific instance
Function<Employee, EmployeeDto> toDto = mapper::toDto;

// In a pipeline
List<EmployeeDto> dtos = employees.stream()
    .map(mapper::toDto)         // same as: e -> mapper.toDto(e)
    .collect(Collectors.toList());
```

```
.forEach(log::info)            // s -> log.info(s)    — log is a specific Logger
.filter(set::contains)         // s -> set.contains(s) — set is a specific Set
.map(prefix::concat)           // s -> prefix.concat(s) — prefix is a specific Stri
```

Method References

- Instance method of an arbitrary instance
  - ClassName::instanceMethod
  - The lambda calls a method on the stream element itself
    - The element is both the receiver and the only input
  - Looks identical to the static method form
    - Both use ClassName::method
  - The compiler distinguishes them by checking whether the method is static or an instance method on the named type

```
// Equivalent lambda
Function<Employee, String> getName = e -> e.name();

// Method reference — Employee is the type; name() is called ON the element
Function<Employee, String> getName = Employee::name;

List<String> names = employees.stream()
    .map(Employee::name)        // same as: e -> e.name()
    .sorted()
    .collect(Collectors.toList());
```

```
.filter(String::isBlank)        // s -> s.isBlank()
.map(Employee::department)      // e -> e.department()
.sorted(String::compareTo)      // (a, b) -> a.compareTo(b)  — two-arg form
```

Method References

- Constructor reference
  - ClassName::new
  - The lambda creates a new instance of the named class
  - The stream element becomes the constructor argument

```
// Given: public record DepartmentDto(String name) {}

// Equivalent lambda
Function<String, DepartmentDto> toDtoL = name -> new DepartmentDto(name);

// Constructor reference
Function<String, DepartmentDto> toDto = DepartmentDto::new;

List<DepartmentDto> dtos = departmentNames.stream()
    .map(DepartmentDto::new)    // same as: name -> new DepartmentDto(name)
    .collect(Collectors.toList());
```

Variable Capture

- Lambdas can reference local variables from the enclosing scope
  - Only if those variables are effectively final
  - Meaning they are never reassigned after the lambda captures them
  - Lambda bodies can outlive their enclosing stack frame
  - A mutable local variable would become unsafe to reference after its frame is gone

```
//  Captured safely — threshold is never reassigned
BigDecimal threshold = configService.getSalaryThreshold();
employees.stream()
    .filter(e -> e.salary().compareTo(threshold) > 0)
    .collect(Collectors.toList());

// Compile error — counter is mutated inside the lambda
int counter = 0;
employees.stream().forEach(e -> counter++);  // COMPILE ERROR

//  Prefer count() for counting; use AtomicInteger only if mutation is genuinely needed
long count = employees.stream().filter(predicate).count();
```

Variable Capture

- When the JVM encounters a lambda
  - It does not execute it immediately
  - It packages the lambda body along with any local variables it references into an object that can be called later
  - That object may be
    - Passed to another method
    - Stored in a field
    - Handed to a different thread
    - Called milliseconds or seconds after the original method has returned
  - By the time the lambda executes, the local variable it captured may no longer exist
    - Local variables live on the stack
    - When a method returns, its stack frame is gone
    - The lambda, however, may still be alive on the heap

```
//  Captured safely — threshold is never reassigned
BigDecimal threshold = configService.getSalaryThreshold();
employees.stream()
    .filter(e -> e.salary().compareTo(threshold) > 0)
    .collect(Collectors.toList());

// Compile error — counter is mutated inside the lambda
int counter = 0;
employees.stream().forEach(e -> counter++);  // COMPILE ERROR

//  Prefer count() for counting; use AtomicInteger only if mutation is genuinely needed
long count = employees.stream().filter(predicate).count();
```

Variable Capture

- Solution
  - Copy the value of the local variable into the lambda object at capture time
  - That copy is safe because it lives on the heap with the lambda, independent of the stack frame
  - This only works if the variable's value does not change after the copy is made
  - If the variable were mutable, the copy inside the lambda would immediately be stale
    - The lambda would see a different value from what the rest of the method sees
    - That is a data race, and it would produce bugs that are nearly impossible to diagnose

```
//  Captured safely — threshold is never reassigned
BigDecimal threshold = configService.getSalaryThreshold();
employees.stream()
    .filter(e -> e.salary().compareTo(threshold) > 0)
    .collect(Collectors.toList());

// Compile error — counter is mutated inside the lambda
int counter = 0;
employees.stream().forEach(e -> counter++);  // COMPILE ERROR

//  Prefer count() for counting; use AtomicInteger only if mutation is genuinely needed
long count = employees.stream().filter(predicate).count();
```

Antipattern

- Side effects in stream lambdas
  - Lambdas in filter, map, and similar operations should be pure functions
  - No external state modification
- Side effects
  - Make the pipeline stateful
  - Break lazy evaluation guarantees
  - Make parallel streams unsafe
- Use forEach deliberately for side effects

```
//  Captured safely — threshold is never reassigned
BigDecimal threshold = configService.getSalaryThreshold();
employees.stream()
    .filter(e -> e.salary().compareTo(threshold) > 0)
    .collect(Collectors.toList());

// Compile error — counter is mutated inside the lambda
int counter = 0;
employees.stream().forEach(e -> counter++);  // COMPILE ERROR

//  Prefer count() for counting; use AtomicInteger only if mutation is genuinely needed
long count = employees.stream().filter(predicate).count();
```

Three Sections of a Pipeline

Section	Description
Source	Creates the stream from a data source: `Collection.stream()`, `Stream.of()`, `Files.lines()`. Does NOT read data — creates a lazy pipeline head.
Intermediate operations	Transform the stream, returning a new stream. Always lazy — no element is processed until a terminal operation is invoked. Multiple operations are fused by the JVM into one pass.
Terminal operation	Consumes the stream and produces a result or side effect. Triggers all computation. Stream is exhausted and cannot be reused.

```
// Nothing executes until collect() is reached
List<EmployeeDto> result = employees.stream()            // 1. Source
    .filter(e -> e.status() == Status.ACTIVE)            // 2. Intermediate (lazy)
    .filter(e -> e.salary().compareTo(THRESHOLD) > 0)    // 2. Intermediate (lazy)
    .sorted(Comparator.comparing(Employee::name))        // 2. Intermediate (lazy)
    .map(employeeMapper::toDto)                          // 2. Intermediate (lazy)
    .collect(Collectors.toUnmodifiableList());           // 3. Terminal — fires everything
```

Intermediate operations on a stream do not execute when they are called. They record what transformation is to be performed. Execution is deferred until a terminal operation is invoked — and only then does data flow through the pipeline.

Operation Fusion

- Recording operations
  - When calling .filter(...) or .map(...) on a stream
  - The JVM does not iterate over a single element, no execution is done
  - It builds an internal description of the operation
    - Essentially a linked list of transformation stages
    - Returns a new stream object that represents the pipeline so far
    - No element from the stream source has been processed

```
Stream<EmployeeDto> pipeline = employees.stream()    // no execution
    .filter(e -> e.status() == Status.ACTIVE)        // no execution – recorded
    .filter(e -> e.salary() > 50_000)                // no execution – recorded
    .map(employeeMapper::toDto);                     // no execution – recorded

// At this point: zero iterations have occurred.
// The pipeline object exists, but no element has been processed.

List<EmployeeDto> result = pipeline.collect(Collectors.toList());
// NOW execution begins. All three recorded operations run together
// in a single pass over the source, element by element.
```

Operation Fusion

- The trigger
  - Execution begins at the exact moment a terminal operation is invoked
    - Terminal operations are the ones that demand a concrete result
    - Collect, count, findFirst, anyMatch, reduce, forEach, etc.
    - They cannot be deferred because they produce a value the calling code immediately needs
- The pipeline
  - Does not run partially
  - It runs exactly once, in full, synchronously, at the point the terminal operation is called

```
Stream<EmployeeDto> pipeline = employees.stream()    // no execution
    .filter(e -> e.status() == Status.ACTIVE)        // no execution – recorded
    .filter(e -> e.salary() > 50_000)                // no execution – recorded
    .map(employeeMapper::toDto);                     // no execution – recorded

// At this point: zero iterations have occurred.
// The pipeline object exists, but no element has been processed.

List<EmployeeDto> result = pipeline.collect(Collectors.toList());
// NOW execution begins. All three recorded operations run together
// in a single pass over the source, element by element.
```

Operation Fusion

- If a stream pipeline has two filters and a map
  - One might reasonably assume the JVM does this
    - Iterate over all elements, apply filter 1, produce a new intermediate list
    - Iterate over that list, apply filter 2, produce another intermediate list
    - Iterate over that list, apply map, produce the final list
  - However, that would be three full passes over the data, plus two throwaway intermediate collections allocated in memory

```
Stream<EmployeeDto> pipeline = employees.stream()    // no execution
    .filter(e -> e.status() == Status.ACTIVE)        // no execution — recorded
    .filter(e -> e.salary() > 50_000)                // no execution — recorded
    .map(employeeMapper::toDto);                     // no execution — recorded

// At this point: zero iterations have occurred.
// The pipeline object exists, but no element has been processed.

List<EmployeeDto> result = pipeline.collect(Collectors.toList());
// NOW execution begins. All three recorded operations run together
// in a single pass over the source, element by element.
```

Operation Fusion

- What actually happens
  - Element by element
  - The unit of execution is one element moving through all stages
  - Not one stage moving through all elements
  - Each element travels as far through the pipeline as it can, then either falls out or reaches the terminal
    - The next element is not fetched until the current one has completed its journey
    - There is no point at which all elements are collected into an intermediate bucket

```
Take employee[0]
  → pass through stage 1 (filter: is ACTIVE?)
      → YES → pass through stage 2 (filter: salary > 50,000?)
          → YES → pass through stage 3 (map: toDto)
              → add result to output list

Take employee[1]
  → pass through stage 1 (filter: is ACTIVE?)
      → NO → stop here. Do not pass to stage 2 or stage 3.

Take employee[2]
  → pass through stage 1 (filter: is ACTIVE?)
      → YES → pass through stage 2 (filter: salary > 50,000?)
          → NO → stop here. Do not pass to stage 3.

... and so on for each element
```

Operation Fusion

- Two concrete benefits
  - No intermediate allocations
    - In the three-pass model, would be two intermediate List objects that are immediately discarded
    - With fusion, those lists never exist.
    - For a source of one million elements, that is two million object allocations saved
  - Early elimination multiplies
    - If stage 1 eliminates 90% of elements, stages 2 and 3 only ever see 10% of the data
    - In a three-pass model, stage 1 still iterates all one million elements
    - In a fused model, stages 2 and 3 are never invoked for the 90% that stage 1 rejects
    - The earlier and more selective the filter, the greater the saving

```
// Source: 1,000,000 employees
// Stage 1 rejects 95% — only 50,000 reach stage 2
// Stage 2 rejects 80% of those — only 10,000 reach stage 3
// Stage 3 (map) runs exactly 10,000 times — not 1,000,000

employees.stream()
    .filter(e -> e.status() == Status.ACTIVE)          // sees 1,000,000 elements
    .filter(e -> e.salary() > 50_000)                  // sees ~50,000 elements
    .map(employeeMapper::toDto)                        // sees ~10,000 elements
    .collect(Collectors.toList());
```

Breaking Fusion

- Fusion applies to stateless intermediate operations
  - Operations that process each element independently
    - Filter and map are stateless
  - sorted() is stateful
    - It cannot produce its first output element until it has seen every input element
    - Because the first sorted element might be the last one in the source
    - This breaks the single-pass model

```
employees.stream()
    .filter(e -> e.status() == Status.ACTIVE)   // fused ┐
    .map(employeeMapper::toDto)                 // fused ┘  one pass up to here
    .sorted(Comparator.comparing(Dto::name))    // BARRIER — must collect all, then sort
    .limit(10)                                  // fused after the sort
    .collect(Collectors.toList());
```

Breaking Fusion

- sorted() acts as a pipeline barrier.
  - Everything before it runs in one fused pass
  - The results are buffered
  - Then sorted() sorts the buffer
  - Then everything after it runs in another fused pass
    - Still get fusion within each segment
    - sorted() forces at least one full materialization of intermediate results
  - Always filter before sorting
    - The filter reduces the number of elements that sorted() has to buffer and sort

```
//  sorted() sees all 1,000,000 elements — buffers them all, then sorts
employees.stream()
    .sorted(Comparator.comparing(Employee::name))
    .filter(e -> e.status() == Status.ACTIVE)
    .collect(Collectors.toList());

//  sorted() sees only the ~50,000 that passed the filter
employees.stream()
    .filter(e -> e.status() == Status.ACTIVE)
    .sorted(Comparator.comparing(Employee::name))
    .collect(Collectors.toList());
```

Fusion Summary

Question	Answer
What is operation fusion?	Multiple intermediate operations executing together as a single traversal of the source — one element at a time through all stages
What does it eliminate?	Intermediate collections between stages, and redundant iterations over already-filtered data
What makes it possible?	Laziness — operations are recorded as a pipeline, not executed immediately, so the JVM can compose them before any element is touched
Which operations fuse?	All stateless intermediates: `filter` , `map` , `flatMap` , `peek` , `mapToInt/Long/Double`
Which operations break fusion?	Stateful intermediates: `sorted()` and `distinct()` — they must see all elements before producing any output
Does the programmer need to do anything?	No — fusion is automatic. The only action required is ordering filters before `sorted()` to minimise what the barrier has to buffer

Stateless vs Stateful

Category	Operations	Implication
Stateless	`filter` , `map` , `flatMap` , `peek` , `mapToInt/Long/Double`	Parallelisation-safe. Each element processed independently. Can be fused with other stateless ops into one pass.
Stateful (ordered)	`sorted` , `distinct`	Must buffer all elements before producing output. `sorted` requires full consumption. `distinct` tracks seen elements across the stream.
Stateful (short-circuit)	`limit` , `skip`	`limit` stops after N. `skip` discards first N. Both interact with ordering guarantees in ordered streams.
Terminal	`collect` , `forEach` , `reduce` , `count` , `findFirst` , `anyMatch` , `allMatch` , `noneMatch` , `toArray` , `sum` , `average` , `min` , `max`	Triggers execution. Consumes the stream. Produces result or side effect.

Stateful Lambda Anti-pattern

- Stream API defines stateless in terms of operations
  - Not the functionality of the lambdas that implement them
  - A filter operation is deemed stateless, regardless of the lambda that implements it
  - That assumption
    - Makes fusion safe
    - Makes parallel execution safe
    - Makes the pipeline predictable

```
//  sorted() sees all 1,000,000 elements — buffers them all, then sorts
employees.stream()
    .sorted(Comparator.comparing(Employee::name))
    .filter(e -> e.status() == Status.ACTIVE)
    .collect(Collectors.toList());

//  sorted() sees only the ~50,000 that passed the filter
employees.stream()
    .filter(e -> e.status() == Status.ACTIVE)
    .sorted(Comparator.comparing(Employee::name))
    .collect(Collectors.toList());
```

Stateful Lambda Anti-pattern

- A stateful lambda
  - A lambda passed to a stateless operation that secretly violates that assumption
    - It consults or modifies something outside the element being processed
    - The operation is still filter
    - The problem is in what the lambda does when it implements the filter
  - Imagine you have a set of employee IDs that have been flagged for exclusion, and you want to filter them out of a stream
    - Shown to the right
    - This compiles and It runs
    - In a simple sequential case with a stable set, it probably produces the right answer
    - The problem is not immediately visible

```
// A shared mutable set — owned and modified elsewhere in the application
Set<Long> excludedIds = new HashSet<>();
excludedIds.add(42L);
excludedIds.add(99L);

// The lambda consults this external set on every invocation
List<Employee> result = employees.stream()
    .filter(e -> !excludedIds.contains(e.id()))   // stateful lambda
    .map(employeeMapper::toDto)
    .collect(Collectors.toList());
```

Stateful Lambda Anti-pattern

- The filter lambda
  - Does not depend only on the stream element
  - Its answer for any given employee depends on the current contents of excludedIds at the moment that specific element is processed

```
// A shared mutable set — owned and modified elsewhere in the application
Set<Long> excludedIds = new HashSet<>();
excludedIds.add(42L);
excludedIds.add(99L);

// The lambda consults this external set on every invocation
List<Employee> result = employees.stream()
    .filter(e -> !excludedIds.contains(e.id()))   // stateful lambda
    .map(employeeMapper::toDto)
    .collect(Collectors.toList());
```

Stateful Lambda Anti-pattern

- The filter lambda
  - Two invocations of the lambda with the same employee may return different results if excludedIds changes between them
  - The result of the pipeline depends on when each element is processed, not just what each element is
  - This is the definition of statefulness
  - The output depends on something other than the immediate input

```
// A shared mutable set — owned and modified elsewhere in the application
Set<Long> excludedIds = new HashSet<>();
excludedIds.add(42L);
excludedIds.add(99L);

// The lambda consults this external set on every invocation
List<Employee> result = employees.stream()
    .filter(e -> !excludedIds.contains(e.id()))   // stateful lambda
    .map(employeeMapper::toDto)
    .collect(Collectors.toList());
```

Stateful Lambda Anti-pattern

- Mutation during iteration
  - If anything modifies excludedIds while the stream is running
    - The pipeline produces inconsistent results
    - Some elements are tested against the old set, some against the new one
  - The output is a mixture of two different filter conditions with no clear boundary between them

```
// A shared mutable set — owned and modified elsewhere in the application
Set<Long> excludedIds = new HashSet<>();
excludedIds.add(42L);
excludedIds.add(99L);

// The lambda consults this external set on every invocation
List<Employee> result = employees.stream()
    .filter(e -> !excludedIds.contains(e.id()))   // stateful lambda
    .map(employeeMapper::toDto)
    .collect(Collectors.toList());
```

Stateful Lambda Anti-pattern

- Parallel streams
  - In a parallel stream, multiple threads invoke the lambda concurrently on different elements, all reading from the same excludedIds
  - HashSet is not thread-safe
    - Concurrent reads are generally safe
    - But if any thread is modifying the set while others are reading it
    - Then the behaviour of HashSet is undefined
  - It can
    - Return incorrect results from contains()
    - Throw a ConcurrentModificationException
    - Infinite-loop internally during a rehash
    - Silently corrupt its own internal structure

```
// A shared mutable set — owned and modified elsewhere in the application
Set<Long> excludedIds = new HashSet<>();
excludedIds.add(42L);
excludedIds.add(99L);

// The lambda consults this external set on every invocation
List<Employee> result = employees.stream()
    .filter(e -> !excludedIds.contains(e.id()))   // stateful lambda
    .map(employeeMapper::toDto)
    .collect(Collectors.toList());
```

Collectors

- Some terminal operations produce a single value
  - count() produces a long
  - findFirst() produces an Optional
  - reduce() produces a single accumulated result
- collect() is different
  - It produces a container
    - A data structure built by processing every element in the pipeline
  - .A Collector is a recipe for building that container specifying
    - How to create the container, the empty structure to start with
    - What to do when a new element arrives
    - How to merge results when parallel streams build multiple containers simultaneously
  - The Collectors utility class is a factory that provides pre-built recipes for the most common containers
    - Rarely need to write a custom Collector, the factory methods cover the overwhelming majority of enterprise use case

The Default Container

- toList() and toUnmodifiableList()
  - Accumulates all stream elements into a List in encounter order
    - Most common thing wanted from a stream pipeline is a list
  - Collectors.toList() returns a mutable ArrayList
    - The caller can add or remove elements
    - This is almost never what is wanted a service query method
  - toList() (Java 16+) returns an unmodifiable list
    - Same as Collectors.toUnmodifiableList()
    - Use toList() as the default

```
List<EmployeeDto> dtos = employees.stream()
    .filter(e -> e.status() == Status.ACTIVE)
    .map(employeeMapper::toDto)
    .collect(Collectors.toList());          // mutable list –

// Preferred from Java 16+
List<EmployeeDto> dtos = employees.stream()
    .filter(e -> e.status() == Status.ACTIVE)
    .map(employeeMapper::toDto)
    .toList();                              // unmodifiable –
```

Sets

- Accumulates elements into a Set
  - Automatically eliminating duplicates
  - Order is not preserved
- Use case
  - Sometimes want the distinct values from a stream, not a list of all values
    - A Set enforces uniqueness structurally
    - Do not need to write deduplication logic
  - When the downstream consumer needs to perform membership tests
    - A Set.contains() check is O(1)
    - A List.contains() check is O(n)

```java
// All unique department codes across all employees
Set<String> departments = employees.stream()
    .map(Employee::department)
    .collect(Collectors.toUnmodifiableSet());

// All unique roles assigned to a specific employee
Set<String> roles = employee.getRoles().stream()
    .map(Role::name)
    .collect(Collectors.toUnmodifiableSet());
```

Building Lookup Structures

- toMap()
  - Accumulates elements into a Map by applying two functions to each element
    - One to produce the key, one to produce the value
- Use case
  - For looking up elements by a key
    - Iterating the list repeatedly is O(n) per lookup
    - Building a Map once is O(n) up front and O(1) per subsequent lookup

```java
// Build a lookup map — id → EmployeeDto
// After this, finding any employee by id is O(1)
Map<Long, EmployeeDto> byId = employees.stream()
    .collect(Collectors.toUnmodifiableMap(
        Employee::id,              // key extractor
        employeeMapper::toDto      // value extractor
    ));

// Usage
EmployeeDto emp = byId.get(42L);   // O(1) — no iteration
```

Building Lookup Structures

- The duplicate key problem
  - toMap() throws IllegalStateException if two elements produce the same key
  - This is intentional
    - A silent overwrite would be a data loss bug
  - If duplicates are possible and expected
    - Must provide a merge function as a third argument that decides which value to keep
  - The three-argument form adds a merge function
    - A BinaryOperator<V> that receives both the existing value and the incoming value whenever a duplicate key is detected, and returns whichever one should be kept

```
// Two employees with the same name — keep the most senior (higher id)
Map<String, Employee> byName = employees.stream()
    .collect(Collectors.toUnmodifiableMap(
        Employee::name,
        e -> e,
        (existing, incoming) -> existing.id() > incoming.id() ? existing : incoming
    ));
```

In-Memory GROUP BY

- Partitions a stream into groups
  - All elements sharing the same key value are collected into the same bucket
  - Produces Map<K, List<V>> by default, where K is the grouping key and V is the element type
  - Use case: any time you need to answer "how many X per Y" or "which X belong to each Y"
  - In-memory equivalent of SQL's GROUP BY clause
    - Supports the same concept of aggregate functions through downstream collectors

```
// Basic grouping — which employees belong to each department?
Map<String, List<Employee>> byDepartment = employees.stream()
    .collect(Collectors.groupingBy(Employee::department));

// Result structure:
// {
//   "Engineering": [Alice, Bob, Carol],
//   "Finance":     [Dave, Eve],
//   "HR":          [Frank]
// }
```

In-Memory GROUP BY

- The downstream collector
  - Applying an aggregate function to each group
  - Once elements are grouped, the downstream collector defines what to do with each group
  - Without a downstream collector, each group is collected into a List
  - With one, any aggregation can be applied

```
// How many employees are in each department?
// SQL equivalent: SELECT department, COUNT(*) FROM employees GROUP BY departme
Map<String, Long> countByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,       // the grouping key
        Collectors.counting()       // downstream: count elements in each group
    ));
// { "Engineering": 3, "Finance": 2, "HR": 1 }

// What is the average salary in each department?
// SQL equivalent: SELECT department, AVG(salary) FROM employees GROUP BY depar
Map<String, Double> avgSalaryByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.averagingDouble(e -> e.salary().doubleValue())
    ));
// { "Engineering": 85000.0, "Finance": 72000.0, "HR": 65000.0 }

// What are the names of employees in each department?
// Downstream collector transforms each element before collecting
Map<String, List<String>> namesByDepartment = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.mapping(Employee::name, Collectors.toList())
    ));
// { "Engineering": ["Alice", "Bob", "Carol"], ... }
```

partitioningBy() - Binary Grouping

- Splits a stream into exactly two groups
  - Those for which the predicate returns true
  - And those for which it returns false
  - Always produces Map<Boolean, List<T>>
- Rationale
  - groupingBy() with a boolean classifier technically does the same thing
  - partitioningBy() exists because binary classification is extremely common
    - Having dedicated syntax makes the intent clearer and the implementation slightly more efficient
    - It avoids hashing the key because there are only two possible keys

```
// Split employees into active and inactive
Map<Boolean, List<Employee>> split = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.status() == Status.ACTIVE));

List<Employee> active   = split.get(true);
List<Employee> inactive = split.get(false);
```

```
Map<Boolean, List<OrderRequest>> validated = orders.stream()
    .collect(Collectors.partitioningBy(orderValidator::isValid));

List<OrderRequest> valid   = validated.get(true);
List<OrderRequest> invalid = validated.get(false);

valid.forEach(orderService::process);
invalid.forEach(auditService::logRejected);
```

partitioningBy() - Binary Grouping

- partitioningBy() also accepts a downstream collector
  - Allows applying an aggregation to each partition rather than collecting into a list

```
// Count how many employees are active vs inactive
Map<Boolean, Long> counts = employees.stream()
    .collect(Collectors.partitioningBy(
        e -> e.status() == Status.ACTIVE,
        Collectors.counting()
    ));
// { true: 847, false: 153 }
```

joining() - Assembling Strings

- Concatenates stream elements into a single String, with an optional delimiter, prefix, and suffix
- Rationale
  - Building a delimited string from a collection is a recurring task
  - Eliminates the need for a StringBuilder loop with awkward handling of the trailing delimiter

```
// Plain concatenation — no delimiter
String allNames = employees.stream()
    .map(Employee::name)
    .collect(Collectors.joining());
// "AliceBobCarol"

// With delimiter
String csv = employees.stream()
    .map(Employee::name)
    .collect(Collectors.joining(", "));
// "Alice, Bob, Carol"

// With delimiter, prefix, and suffix
String formatted = employees.stream()
    .map(Employee::name)
    .collect(Collectors.joining(", ", "[", "]"));
// "[Alice, Bob, Carol]"
```

counting()

- Counts the number of elements that reach it
  - Returns a Long
- Rationale
  - As a standalone terminal operation, stream().count() does the same thing more simply
  - counting() exists as a downstream collector
    - To count elements within each group produced by groupingBy() or partitioningBy()

```
// Standalone counting — just use count()
long activeCount = employees.stream()
    .filter(e -> e.status() == Status.ACTIVE)
    .count();

// Downstream counting — this is where counting() is actually useful
Map<String, Long> countByDepartment = employees.stream()
    .collect(Collectors.groupingBy(Employee::department, Collectors.counting()));
```

Summarizing

- Multiple output forms
  - summarizingDouble()
  - summarizingInt()
  - summarizingLong()
  - Computes count, sum, min, max, and average in a single pass
  - Returns a DoubleSummaryStatistics or equivalent object that exposes all five values
  - Also used as a downstream collector

```
// Five aggregates in a single pass — no repeated iteration
DoubleSummaryStatistics salaryStats = employees.stream()
    .collect(Collectors.summarizingDouble(e -> e.salary().doubleValue()));

long   count   = salaryStats.getCount();    // how many employees
double total   = salaryStats.getSum();      // total payroll
double average = salaryStats.getAverage();  // mean salary
double minimum = salaryStats.getMin();      // lowest salary
double maximum = salaryStats.getMax();      // highest salary
```

```
// Salary statistics per department — all five figures, all groups, one stream pass
Map<String, DoubleSummaryStatistics> statsByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.summarizingDouble(e -> e.salary().doubleValue())
    ));

// Access any figure for any department
double engineeringAvg = statsByDept.get("Engineering").getAverage();
double financeMax     = statsByDept.get("Finance").getMax();
```

Single Numeric Aggregates

- Two forms
  - averagingDouble()
  - summingDouble()
  - Computes a single numeric aggregate, mean or sum, and returns a Double

```
// Average salary across all employees
double avgSalary = employees.stream()
    .collect(Collectors.averagingDouble(e -> e.salary().doubleValue()));

// Total payroll
double totalPayroll = employees.stream()
    .collect(Collectors.summingDouble(e -> e.salary().doubleValue()));
```

```
// Average salary per department
Map<String, Double> avgByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.averagingDouble(e -> e.salary().doubleValue())
    ));
```

Transforming Before Collecting

- Applies a mapping function to each element before passing it to a downstream collector
  - Collector equivalent of .map() in a pipeline
  - Cannot insert a .map() step inside groupingBy()
    - The grouping has already happened
    - mapping() lets you apply a transformation within the collection phase

```
// Without mapping() — groups contain Employee objects
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::department));

// With mapping() — groups contain names (Strings), not Employee objects
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.mapping(Employee::name, Collectors.toList())
    ));
// { "Engineering": ["Alice", "Bob"], "Finance": ["Dave"] }
```

Post-Processing the Result

- collectingAndThen()
  - Applies a finishing function to the result of another collector after it has finished accumulating.
  - Sometimes the container produced by a collector is not quite the shape needed
  - collectingAndThen() lets you transform it without an extra stream pass

```
// Collect to a list, then wrap it in an unmodifiable view
List<EmployeeDto> dtos = employees.stream()
    .map(employeeMapper::toDto)
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        Collections::unmodifiableList
    ));

// Collect to a map, then extract just the values as a list
List<Employee> deduplicated = employees.stream()
    .collect(Collectors.collectingAndThen(
        Collectors.toMap(Employee::id, e -> e, (a, b) -> a),
        map -> new ArrayList<>(map.values())
    ));
```

Collector Summary

- The downstream collector pattern
  - Passing one Collector as an argument to another allows these building blocks to compose
  - groupingBy() + counting()
    - Answers "how many per group"
  - groupingBy() + mapping() + toList()
    - Answers "which transformed values belong to each group"
  - Similar to the pipe and filter architecture of UNIX utilities

Collector	Question it answers
`toList()` / `toUnmodifiableList()`	Give me all elements in a list
`toSet()`	Give me the distinct elements in a set
`toMap()`	Give me a lookup structure keyed by some field
`groupingBy()`	Give me the elements organised into groups by some classification
`partitioningBy()`	Give me the elements split into two groups by a yes/no question
`joining()`	Give me all elements concatenated into a single string
`counting()`	Tell me how many elements there are
`summarizingDouble()`	Tell me the count, sum, min, max, and average of a numeric field
`averagingDouble()`	Tell me the average of a numeric field
`summingDouble()`	Tell me the sum of a numeric field
`mapping()`	Transform each element before collecting
`collectingAndThen()`	Collect, then transform the result

Primitive Streams

- Stream<T> is generic
  - Stream<Integer> stores Integer objects in a boxed format
  - Boxing and unboxing on every element has measurable overhead for large numeric collections
- IntStream, LongStream, DoubleStream
  - Operate on primitives directly without boxing
  - Also support sum(), average(), min(), max(), and summaryStatistics() without a Collector

```
OptionalDouble average = IntStream.rangeClosed(1, 50)  // source
    .filter(n -> n % 3 == 0)                           // divisible by 3
    .map(n -> n * 2)                                   // double them
    .limit(5)                                          // take first 5: 6,12,18,24,3
    .peek(n -> System.out.println("Processing: " + n)) // debug/logging
    .average();                                        // terminal op

average.ifPresent(a -> System.out.println("Average: " + a)); // 18.0
```

```
int result = IntStream.rangeClosed(1, 20)   // 1, 2, 3, ... 20
    .filter(n -> n % 2 != 0)                // 1, 3, 5, ... 19
    .map(n -> n * n)                        // 1, 9, 25, ... 361
    .sum();                                 // 1000

System.out.println(result); // 1000
```

Understanding flatMap

- map() applies a function to each element and collects the results
  - It always produces one output element per input element
  - The output stream has exactly the same number of elements as the input stream
  - This works perfectly when your transformation produces a single value
  - It breaks down when your transformation produces a collection

```
// Each Department contains a List<Employee>
// map applies getDepartment.employees() to each department
// Result: one List<Employee> per department — wrapped inside the stream

Stream<List<Employee>> nested = departments.stream()
    .map(Department::employees);

// The stream now contains Lists, not Employees:
// [ [Alice, Bob], [Carol], [Dave, Eve, Frank] ]
//      ↑              ↑           ↑
//   List 1         List 2      List 3
// Three lists inside a stream — not what we wanted
```

Understanding flatMap

- flatMap() does two things in sequence that together solve the nesting problem
  - Map: apply the function to each element, producing a stream per element
  - Flatten: merge all those individual streams into one single stream
- The flat in flatMap refers to this flattening step
  - The nested structure is collapsed into a single level

```
// flatMap applies the function AND flattens the result
List<Employee> allEmployees = departments.stream()
    .flatMap(dept -> dept.employees().stream())
    .collect(Collectors.toList());

// What happens element by element:
//
// dept=Engineering → dept.employees().stream() → Stream[Alice, Bob]     ┐
// dept=Finance     → dept.employees().stream() → Stream[Carol]           ├→ Stream
// dept=HR          → dept.employees().stream() → Stream[Dave, Eve, Frank]┘
//
// Three separate streams are merged into one flat stream
// Six employees, no wrapping lists, ready to process
```

Understanding flatMap

- The difference between map and flatMap is entirely about the shape of what the function returns

If your function returns...	Use...	Result shape
A single transformed value	`map`	One output element per input element
A collection or stream of values	`flatMap`	All values from all collections, merged into one stream

```
// ── map: function returns a single value ──────────────────────────

// String is a single value — map is correct
Stream<String> names = employees.stream()
    .map(Employee::name);              // one name per employee

// ── map: function returns a collection — this is the problem case ─────

// List<String> is a collection — map wraps it, producing nested structure
Stream<List<String>> nested = employees.stream()
    .map(Employee::skills);            // one List<String> per employee  (nested)

// ── flatMap: function returns a stream — unwraps and merges ────────────

// flatMap converts each List to a stream, then merges all streams
Stream<String> allSkills = employees.stream()
    .flatMap(e -> e.skills().stream()); // all skills from all employees
```

Understanding flatMap

- The missing .stream() call
  - FlatMap() must return a Stream, not a List
  - The most common mistake is writing
    - .flatMap(e -> e.skills())
  - which does not compile because skills() returns a List, not a Stream
  - The correct form is
    - .flatMap(e -> e.skills().stream())
  - The .stream() call is always required to convert the collection into a stream that flatMap can merge

```
// ── map: function returns a single value ──────────────────────────

// String is a single value — map is correct
Stream<String> names = employees.stream()
    .map(Employee::name);              // one name per employee

// ── map: function returns a collection — this is the problem case ──

// List<String> is a collection — map wraps it, producing nested structure
Stream<List<String>> nested = employees.stream()
    .map(Employee::skills);            // one List<String> per employee  (nested)

// ── flatMap: function returns a stream — unwraps and merges ─────────

// flatMap converts each List to a stream, then merges all streams
Stream<String> allSkills = employees.stream()
    .flatMap(e -> e.skills().stream()); // all skills from all employees
```

Understanding Optional

- A method's return type does indicate whether null is a possible return value
  - Especially when waiting for a response from another thread, which may die or never return, effectively a null return
- The options for handling this case are shown in the example
- None of the options solve the problem

```
// What does this method return when the employee is not found?
// The type signature gives no information.
Employee findByEmail(String email);    // returns null? throws? returns a default?
```

The caller has three options, all bad:

```
// Option 1 — assume it never returns null. Wrong assumption → NullPointerException in production.
String dept = employeeService.findByEmail(email).department();

// Option 2 — defensively null-check everything. Verbose, easy to forget one.
Employee e = employeeService.findByEmail(email);
if (e != null) {
    String dept = e.department();
}

// Option 3 — check the documentation. Documentation goes stale.
// @return the employee, or null if not found  ← easy to miss, not enforced
```

Understanding Optional

- Optional<T> is a container object
  - It holds either one value of type T, or nothing
  - It is a typed signal that says
  - "the value here may or may not be present, and I am forcing you to deal with that before you can access it"

```
// A method that returns Optional declares its contract in its signature
Optional<Employee> findByEmail(String email);
//       ↑
//  "This may return nothing. Handle both cases."

// The compiler now enforces that the caller addresses the absent case.
// You cannot call .department() on an Optional<Employee> directly —
// you must unwrap it first, and unwrapping forces the decision.
```

```
// Present — contains a value
Optional<Employee> present = Optional.of(new Employee(42L, "Alice"));
present.isPresent();    // true
present.get();          // Employee(42, "Alice")

// Empty — contains nothing
Optional<Employee> empty = Optional.empty();
empty.isPresent();      // false
empty.get();            // NoSuchElementException — this is why you never call get() directly
```

```
Optional.of(value)          // value must not be null — throws NullPointerException if it is
Optional.ofNullable(value)  // value may be null — produces empty Optional if null
Optional.empty()            // explicitly empty — use in methods that return no result
```

The Functional API

- Optional provides a functional API
  - Use case is to handle both the present and absent cases
    - Without ever calling get() directly
    - Without writing if (optional.isPresent()) checks that replicate the null-check pattern
  - Options
    - Transforming the value if present: map()
    - Providing a default when absent: orElse() and orElseGet()
    - Throwing when absence is an error: orElseThrow()

```
// Without Optional — null check required before transformation
Employee employee = findByEmail(email);  // may be null
String dept = employee != null ? employee.department() : null;

// With Optional — map applies the function only if a value is present
// If empty, map returns another empty Optional — the absence propagates
Optional<String> dept = findByEmail(email).map(Employee::department);
```

```
// orElse — return this value if the Optional is empty
String dept = findByEmail(email)
    .map(Employee::department)
    .orElse("Unassigned");          // default if no employee found

// orElseGet — compute the default lazily (only if actually empty)
String dept = findByEmail(email)
    .map(Employee::department)
    .orElseGet(() -> configService.getDefaultDepartment());  // not called if prese
```

The Functional API

- Optional provides a functional API
  - Use case is to handle both the present and absent cases
    - Without ever calling get() directly
    - Without writing if (optional.isPresent()) checks that replicate the null-check pattern
  - Options
    - Transforming the value if present: map()
    - Providing a default when absent: orElse() and orElseGet()
    - Throwing when absence is an error: orElseThrow()

```
// The canonical service-layer pattern:
// absence is not a normal case — it is an error that should propagate
public EmployeeDto findById(Long id) {
    return employeeRepository.findById(id)
        .map(employeeMapper::toDto)
        .orElseThrow(() -> new EntityNotFoundException("Employee " + id + " not found"));
}
```

Optional Use

- Optional is the right tool in specific situations
  - Overusing it adds indirection and complexity without adding safety
- Use Optional when
  - Method return type
    - When absence is a normal possible outcome
  - Intermediate values in a chain
    - When transforming a value that may be absent
  - Stream terminal operations that may not find a result

```
//  Method return type — when absence is a normal possible outcome
Optional<Employee> findByEmail(String email);
Optional<Employee> findMostRecentLeaver();

//  Intermediate values in a chain — when transforming a value that may be absent
Optional<String> dept = findByEmail(email).map(Employee::department);

//  Stream terminal operations that may not find a result
Optional<Employee> highestPaid = employees.stream()
    .max(Comparator.comparing(Employee::salary));
```

Optional Use

- Do not use Optional for
  - Method parameter
    - Forces callers to wrap values
    - Use overloads instead
  - Collection elements
    - Use filter() to remove absent values instead
  - JPA entity fields
    - Causes serialization problems
  - When the method always returns a value
    - Do not wrap unnecessarily

```
//  Method parameters — forces callers to wrap values; use overloads instead
public void process(Optional<String> name) { ... }  // wrong
public void process(String name) { ... }            // correct

//  Collection elements — use filter() to remove absent values instead
List<Optional<Employee>> list = ...;   // wrong — awkward to work with
List<Employee> list = ...;             // correct — filter before collecting

//  JPA entity fields — causes serialisation problems
@Entity
public class Employee {
    private Optional<String> middleName;   // wrong — use @Column(nullable=true) instead
}

//  When the method always returns a value — do not wrap unnecessarily
Optional<List<Employee>> findAll() { ... }   // wrong — return empty list, not empty Optional
List<Employee> findAll() { ... }             // correct
```

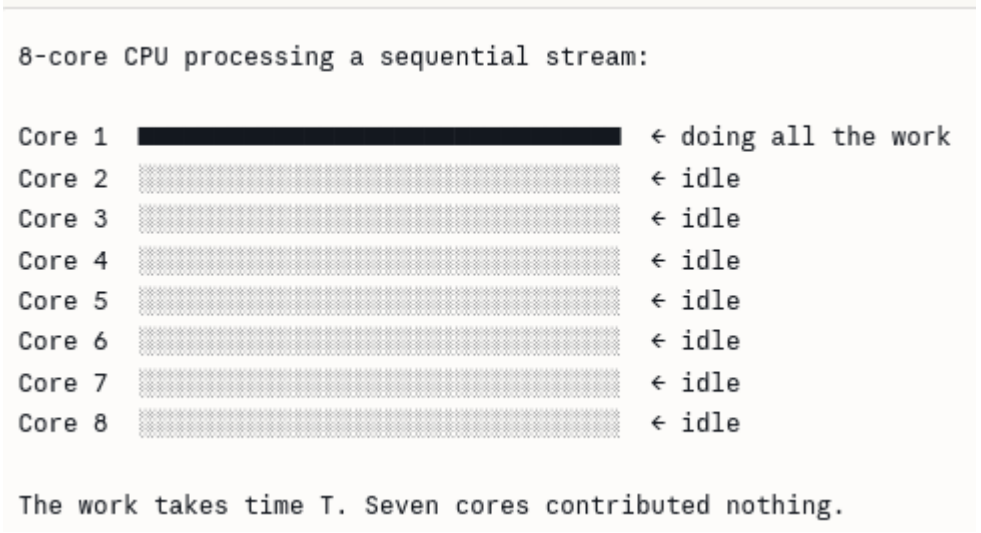
Optional Use

- Table provides a list of times to use Optional

Method	When to Use
`map(Function<T,U>)`	Transform if present → `Optional<U>`. The primary composition tool.
`flatMap(Function<T,Optional<U>>)`	When the mapper itself returns `Optional`. Avoids `Optional<Optional<T>>`.
`filter(Predicate<T>)`	Return empty if value is present but fails the predicate.
`orElse(T other)`	Return value or default. `other` is always evaluated — avoid if construction is expensive.
`orElseGet(Supplier<T>)`	Return value or lazily compute default. Preferred over `orElse` for expensive defaults.
`orElseThrow(Supplier<X>)`	Return value or throw. The correct way to propagate not-found errors from service methods.
`ifPresent(Consumer<T>)`	Side effect if present. Use for logging, event publishing.
`ifPresentOrElse(Consumer, Runnable)`	Action if present, action if absent. Replaces nested `if (opt.isPresent())` blocks.
`stream()`	Convert to `Stream<T>` of 0 or 1 elements. Enables `flatMap(Optional::stream)` pattern.

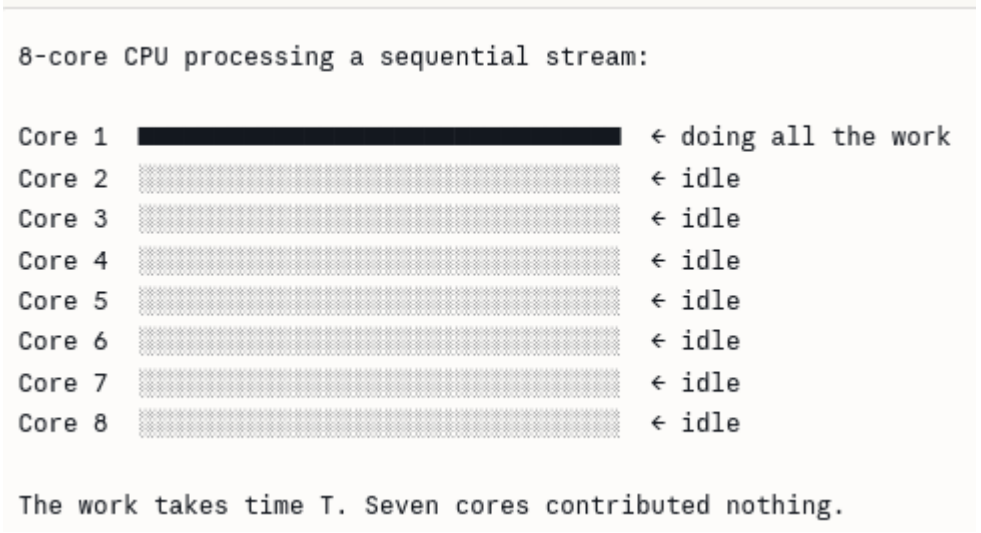
Parallel Streams

- A modern server has many CPU cores
  - Each core can execute instructions independently and simultaneously
  - A standard sequential stream uses exactly one of those cores
  - For CPU-intensive work on large collections, this is a genuine inefficiency
    - If the work can be divided into independent pieces
    - Each core could process a portion simultaneously, and the total time would drop toward T/8
  - Given a collection processing task
    - Can the JVM divide the elements among available cores and recombine the results automatically
    - Without the programmer writing any threading code



Parallel Streams

- Parallelism
  - Making idle cores do work is only worthwhile if the work is the bottleneck
  - If the operation is I/O-bound then more cores do not help
    - The cores are not idle because there is no work
    - They are idle because the work is waiting for something external
  - Parallel streams address CPU-bound bottlenecks only



The Fork/Join Model

- Calling .parallelStream() or .parallel(),
  - Tells the JVM to use a structured decomposition strategy called fork/join
- Three phase strategy
- Split (Fork)
  - The source collections is divided into chunks using a “Spliterator”
    - For an ArrayList of one million elements on an 8-core machine, it might split into 8 chunks of roughly 125,000 elements each
  - The splitting is recursive
    - Each chunk may be split further until chunks are small enough to process efficiently

```
Source: [e1, e2, e3, e4, e5, e6, e7, e8]  (simplified to 8 elements)

Phase 1 — Split:
  Chunk A: [e1, e2]   → submitted to Core 1
  Chunk B: [e3, e4]   → submitted to Core 2
  Chunk C: [e5, e6]   → submitted to Core 3
  Chunk D: [e7, e8]   → submitted to Core 4

Phase 2 — Process (all simultaneous):
  Core 1: filter(e1) map(e1), filter(e2) map(e2) → [dto1, dto2]
  Core 2: filter(e3) map(e3), filter(e4)         → [dto3]
  Core 3: filter(e5) map(e5), filter(e6) map(e6) → [dto5, dto6]
  Core 4: filter(e7)          filter(e8) map(e8)  → [dto8]

Phase 3 — Combine:
  [dto1, dto2] + [dto3] + [dto5, dto6] + [dto8] → [dto1, dto2, dto3, dto5, dto6,
dto8]
```

The Fork/Join Model

- Process
  - Each chunk is submitted to a thread in the common ForkJoinPool as an independent task
  - Each thread applies the full pipeline to its own chunk
  - Done independently, with no coordination required between threads
  - This is where the parallel speedup occurs

```
Source: [e1, e2, e3, e4, e5, e6, e7, e8]  (simplified to 8 elements)

Phase 1 — Split:
  Chunk A: [e1, e2]   → submitted to Core 1
  Chunk B: [e3, e4]   → submitted to Core 2
  Chunk C: [e5, e6]   → submitted to Core 3
  Chunk D: [e7, e8]   → submitted to Core 4

Phase 2 — Process (all simultaneous):
  Core 1: filter(e1) map(e1), filter(e2) map(e2) → [dto1, dto2]
  Core 2: filter(e3) map(e3), filter(e4)         → [dto3]
  Core 3: filter(e5) map(e5), filter(e6) map(e6) → [dto5, dto6]
  Core 4: filter(e7)          filter(e8) map(e8)  → [dto8]

Phase 3 — Combine:
  [dto1, dto2] + [dto3] + [dto5, dto6] + [dto8] → [dto1, dto2, dto3, dto5, dto6,
dto8]
```

The Fork/Join Model

- Combine (Join)
  - The results from each thread are combined into a single final result
  - For toList(), the partial lists are concatenated
  - For count(), the partial counts are summed
  - The Collector defines how this combination works

```
Source: [e1, e2, e3, e4, e5, e6, e7, e8]  (simplified to 8 elements)

Phase 1 — Split:
  Chunk A: [e1, e2]   → submitted to Core 1
  Chunk B: [e3, e4]   → submitted to Core 2
  Chunk C: [e5, e6]   → submitted to Core 3
  Chunk D: [e7, e8]   → submitted to Core 4

Phase 2 — Process (all simultaneous):
  Core 1: filter(e1) map(e1), filter(e2) map(e2) → [dto1, dto2]
  Core 2: filter(e3) map(e3), filter(e4)         → [dto3]
  Core 3: filter(e5) map(e5), filter(e6) map(e6) → [dto5, dto6]
  Core 4: filter(e7)          filter(e8) map(e8)  → [dto8]

Phase 3 — Combine:
  [dto1, dto2] + [dto3] + [dto5, dto6] + [dto8] → [dto1, dto2, dto3, dto5, dto6,
dto8]
```

The Fork/Join Model

- Caveat
  - Not all collections split equally well
  - ArrayList and arrays split perfectly
    - The spliterator knows the size upfront and can divide at any index
  - LinkedList splits poorly
    - Must traverse the list to find the midpoint
  - HashSet splits reasonably
  - The quality of the split directly affects how evenly work is distributed across cores
  - Uneven splits mean some cores finish early and sit idle while others are still processing, reducing the parallelism benefit

```
Source: [e1, e2, e3, e4, e5, e6, e7, e8]  (simplified to 8 elements)

Phase 1 — Split:
  Chunk A: [e1, e2]   → submitted to Core 1
  Chunk B: [e3, e4]   → submitted to Core 2
  Chunk C: [e5, e6]   → submitted to Core 3
  Chunk D: [e7, e8]   → submitted to Core 4

Phase 2 — Process (all simultaneous):
  Core 1: filter(e1) map(e1), filter(e2) map(e2) → [dto1, dto2]
  Core 2: filter(e3) map(e3), filter(e4)         → [dto3]
  Core 3: filter(e5) map(e5), filter(e6) map(e6) → [dto5, dto6]
  Core 4: filter(e7)          filter(e8) map(e8)  → [dto8]

Phase 3 — Combine:
  [dto1, dto2] + [dto3] + [dto5, dto6] + [dto8] → [dto1, dto2, dto3, dto5, dto6,
dto8]
```

The Fork/Join Model

- ForkJoinPool
  - Parallel streams do not create their own thread
  - They submit tasks to the common ForkJoinPool
  - This is a single, shared, JVM-wide thread pool that exists independently of any application code
  - The common ForkJoinPool is not private to parallel streams
    - It is shared across the entire JVM process

```
// Default pool size = number of available CPU cores minus one
// The minus one reserves one core for the calling thread
int poolSize = Runtime.getRuntime().availableProcessors() - 1;

// On a 4-core machine:  pool has 3 threads
// On an 8-core machine: pool has 7 threads
// On a 16-core machine: pool has 15 threads

// Can be overridden with a system property — but rarely should be
System.setProperty("java.util.concurrent.ForkJoinPool.common.parallelism", "4");
```

```
Common ForkJoinPool (7 threads on 8-core machine)
        ↑                       ↑                    ↑
Parallel streams    CompletableFuture       Other libraries
                    .supplyAsync()          using fork/join
                    (without custom
                     executor)
```

The Fork/Join Model

- The saturation problem
  - If a parallel stream submits 7 tasks where each blocks waiting for a database response
  - All 7 pool threads are occupied and blocked
  - The pool is saturated
  - Any other code that needs the pool now finds no threads available so it queues
  - Everything depending on the pool grinds to a halt

```
Scenario: parallel stream calls repository inside each lambda

Thread 1: waiting for DB query... (blocked)
Thread 2: waiting for DB query... (blocked)
Thread 3: waiting for DB query... (blocked)
Thread 4: waiting for DB query... (blocked)
Thread 5: waiting for DB query... (blocked)
Thread 6: waiting for DB query... (blocked)
Thread 7: waiting for DB query... (blocked)

CompletableFuture.supplyAsync(() -> ...) → no threads available → queued
Other parallel stream → no threads available → queued
Application appears hung
```

Parallelism

- Four conditions must all be true for parallelism to improve performance
- Work must be CPU-bound
  - The operation must consume CPU cycles, not wait for something external
  - Computation, transformation, mathematical calculation are CPU-bound.
  - Database queries, HTTP calls, file reads are I/O-bound
  - Parallelism only helps CPU-bound work

```
//  CPU-bound — mathematical computation on each element
double result = largeList.parallelStream()
    .mapToDouble(item -> Math.pow(item.value(), 2) + Math.sqrt(item.weight()))
    .sum();

//  I/O-bound — waits for database on each element
List<Dto> result = ids.parallelStream()
    .map(repository::findById)    // blocks on I/O — parallelism provides no benefit,
    .collect(Collectors.toList()); // and saturates the ForkJoinPool
```

Parallelism

- The collection must be large enough
  - Splitting the collection, submitting tasks to the pool, and combining results are overheads
  - For small collections, these overhead cost more time than the parallelism saves
  - The break-even point depends on the hardware and the operation
  - A practical rule: collections below roughly 10,000 elements rarely benefit from parallelism

```
// Too small — coordination overhead exceeds any parallelism benefit
List<String> tiny = List.of("Alice", "Bob", "Carol");
tiny.parallelStream().map(String::toUpperCase).toList();  // slower than sequential
```

Parallelism

- Lambdas must be stateless and independent
  - Each element must be processable without knowledge of any other element and without modifying shared state
  - If lambda invocations depend on each other or share mutable data, parallelism produces incorrect results
- The source must split well
  - ArrayList, arrays, and IntStream.range() split efficiently
  - LinkedList, Iterator-based sources, and custom collections may not
  - Poor splitting means uneven work distribution and cores sitting idle

```
//  Stateful — shared mutable accumulator causes data corruption in parallel
List<String> results = new ArrayList<>();
employees.parallelStream()
    .forEach(e -> results.add(e.name()));  // ArrayList is not thread-safe
                                           // concurrent add() corrupts the list
```

Encounter Order

- Sequential streams
  - Process elements in encounter order
  - This is the order they appear in the source
    - For a List, that is insertion order
    - For a Set, it is whatever order the set happens to iterate in
- Parallel streams
  - Divide the source into chunks processed by different threads
  - The threads finish at different times in an unpredictable order
  - The final result may not preserve the encounter order of the source

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8);

// Sequential — always [1, 2, 3, 4, 5, 6, 7, 8]
numbers.stream()
    .map(n -> n * 2)
    .toList();

// Parallel — may produce [5, 6, 1, 2, 7, 8, 3, 4] or any other ordering
numbers.parallelStream()
    .map(n -> n * 2)
    .toList();
```

Encounter Order

- The practical implication
  - If the order of results matters to the caller
  - Use a sequential stream, or add an explicit sorted() after the parallel processing
  - For example
    - For display, for pagination, for audit trails
  - Do not rely on parallelStream() to preserve order. encounter order of the source

Situation	Order preserved?
Source is an ordered collection (`List` , array) + `toList()`	Yes — framework enforces order at the cost of some overhead
Source is an ordered collection + `forEach()`	No — `forEach` does not guarantee order in parallel
Source is unordered (`Set` , `Stream.generate()`)	No — no encounter order to preserve
`forEachOrdered()` used instead of `forEach()`	Yes — but eliminates most parallelism benefit
`findFirst()`	Yes — returns first in encounter order
`findAny()`	No — returns whichever element a thread finds first

Encounter Order

- The practical implication
  - If the order of results matters to the caller
  - Use a sequential stream, or add an explicit sorted() after the parallel processing
  - For example
    - For display, for pagination, for audit trails
  - Do not rely on parallelStream() to preserve order. encounter order of the source

Situation	Order preserved?
Source is an ordered collection (`List` , array) + `toList()`	Yes — framework enforces order at the cost of some overhead
Source is an ordered collection + `forEach()`	No — `forEach` does not guarantee order in parallel
Source is unordered (`Set` , `Stream.generate()`)	No — no encounter order to preserve
`forEachOrdered()` used instead of `forEach()`	Yes — but eliminates most parallelism benefit
`findFirst()`	Yes — returns first in encounter order
`findAny()`	No — returns whichever element a thread finds first

Questions?